Design Track

Interview Preparation Series • Professional Edition

Low-Level Design

06

Compiled with LaTeX • June 15, 2026

harsha.somisetty01@gmail.com

In This Edition

1 Low-Level Design (LLD) / Object-Oriented Design 2

1 Low-Level Design (LLD) / Object-Oriented Design

1.1 Comprehensive Interview-Prep Guide

> **Target**: FAANG interviews requiring class-level design, OOP mastery, and design pattern fluency. **Format**: Deep learning + quick-revision sections. Revisit the cheat sheet before every interview.

1.2 Overview --- What It Is

Low-Level Design (LLD) is the art of translating a feature requirement into a concrete, maintainable class structure. Where High-Level Design (HLD) asks *"what boxes talk to each other?"*, LLD asks *"what does each box look like inside?"*.

LLD covers:

- **Class hierarchies** — what objects exist, what they know, what they do
- **Object relationships** — how objects collaborate (inheritance, composition, association)
- **Design patterns** — proven recipes for recurring structural problems
- **SOLID / clean-code principles** — rules that keep code changeable over years
- **UML notation** — a shared visual language to express designs without writing code

Think of LLD as **architecture at the function and class level**, just like an engineer laying out rooms inside a building (vs. HLD which decides the city block).

1.3 Why It Exists

Software without deliberate design rots. Symptoms of bad LLD:

- Every new feature requires touching dozens of classes (**shotgun surgery**)
- Adding a discount type breaks the payment class (**tight coupling**)
- Copy-pasted logic across modules that diverge silently (**DRY violations**)
- Unit tests require 20 mocks (**low cohesion**)

LLD exists to produce code that is:

Goal	What It Means
Extensible	Add features by adding code, not changing existing code
Maintainable	A new engineer understands a class in < 5 minutes
Testable	Classes have clear contracts; can be tested in isolation
Reusable	Components are general enough to appear in multiple contexts
Readable	Code reads like prose; intent is obvious

1.4 Why FAANG Cares

Amazon — Bar raisers explicitly look for OOD clarity. Leadership Principle "Insist on Highest Standards" maps directly to clean class design. Expect to design systems like a parking lot, library, or food-ordering system end-to-end.

Google — Evaluates *code quality* as a separate signal. Interviewers check: Are abstractions at the right level? Does the candidate know when NOT to use a pattern?

Meta — "Crush it" culture means shipping fast without accumulating debt. LLD tests whether you write code that won't become legacy in 6 months.

Microsoft — Heavy on SOLID and extensibility. Will ask you to extend your own design with a new requirement mid-interview.

Apple — Cares about correct ownership semantics (composition vs. inheritance) and interface design.

Specific signals they're testing:

1. Do you naturally think in **nouns (entities) and verbs (behaviors)**?
2. Can you **draw a class diagram on a whiteboard** in 10 minutes?
3. Do you know **when a design pattern applies** vs. when it's overkill?
4. Can you **articulate trade-offs** (e.g., "I used Strategy here instead of a giant if-else so adding new algorithms doesn't touch existing code")?
5. Do you write **clean, idiomatic code** with meaningful names?

1.5 The LLD Interview Framework

Follow this seven-step process in every LLD interview. It signals structure and prevents blank-page panic.

Step 1: Clarify Requirements (2--3 min)

Ask functional and constraint questions:

- "Should we support multiple floors in the parking lot?"
- "Is the elevator system for one building or configurable?"
- "Do we need persistence or just in-memory?"
- "What scale — one elevator or N?"

Goal: Narrow scope. You can't design what you don't understand.

Step 2: Identify Entities / Nouns (2 min)

Extract nouns from the requirements. Each noun is a candidate class.

> "Design a Parking Lot" → ParkingLot, Floor, ParkingSpot, Vehicle, Ticket, Payment, Gate, Attendant

Goal: Build your initial class inventory.

Step 3: Identify Relationships (2 min)

For each pair of entities, ask:

- Is one a *kind-of* another? → **Inheritance**
- Does one *contain/own* another? → **Composition**
- Does one *use* another? → **Association / Dependency**

Goal: Draw arrows. Sketch rough UML.

Step 4: Class Diagram (3--5 min)

Sketch on whiteboard:

- Classes with key **attributes** (fields) and **methods**
- Access modifiers (+ public, - private, # protected)
- Relationships (arrows)

Say aloud: *"I'll keep this diagram minimal now and fill in details when I code."*

Step 5: Apply Patterns (1--2 min)

Scan your diagram for pattern opportunities:

- Multiple vehicle types but same interface? → **Strategy** or **Template Method**
- Need one ParkingLot instance? → **Singleton**
- Create vehicles without knowing exact type? → **Factory**
- Notify displays when spot availability changes? → **Observer**

Don't force patterns. Say: *"I'm considering Strategy here because the ticket pricing algorithm may vary."*

Step 6: Code Core Classes (10--15 min)

Code the most important 2–3 classes fully. Show:

- Constructors, getters/setters (or lack thereof — explain why)
- Core business methods
- Use of patterns where you flagged them

Prioritize **readability over completeness**.

Step 7: Discuss Extensibility (2--3 min)

Proactively say:

- ”If we add electric vehicle spots, I'd extend ParkingSpot with an EVSpot subclass without touching existing code — that's OCP.”
- ”If pricing changes to subscription-based, we swap in a new PricingStrategy.”

This step separates senior candidates from junior ones.

1.6 Core Concepts

OOP Four Pillars

Encapsulation **Definition**: Bundle data and the methods that operate on it; hide internal state; expose only a controlled interface.

Why: Protects invariants. A `BankAccount` shouldn't let outsiders directly set `balance = -1000`.

CODE

```
# BAD — no encapsulation
class BankAccount:
    balance = 0  # public field; anyone can set it

# GOOD — encapsulated
class BankAccount:
    def __init__(self, initial: float):
        self._balance = initial  # private

    def deposit(self, amount: float):
        if amount <= 0:
            raise ValueError("Must be positive")
        self._balance += amount

    @property
    def balance(self) -> float:
        return self._balance
```

Interview Takeaway: Encapsulation = *”Tell, don't ask.”* Objects manage their own state; callers invoke behavior, not data.

Abstraction **Definition**: Expose *what* an object does, not *how* it does it. Hide implementation details behind an interface or abstract class.

CODE

```
from abc import ABC, abstractmethod

class PaymentProcessor(ABC):          # abstraction
    @abstractmethod
    def process(self, amount: float) -> bool:
        ...

class StripeProcessor(PaymentProcessor):  # concrete detail hidden
```

```
    def process(self, amount: float) -> bool:
        # Stripe API calls hidden here
        return True
```

Interview Takeaway: Abstraction lets you swap implementations (Stripe → PayPal) without touching calling code. Think *interface contracts*.

Inheritance **Definition**: A class (child) inherits attributes and behaviors of another class (parent), extending or overriding them.

```
class Vehicle:
    def __init__(self, make: str, speed: int):
        self.make = make
        self.speed = speed

    def move(self) -> str:
        return f"{self.make} moves at {self.speed} km/h"

class ElectricCar(Vehicle):
    def __init__(self, make: str, speed: int, battery_kwh: int):
        super().__init__(make, speed)
        self.battery_kwh = battery_kwh

    def move(self) -> str:           # override
        return f"{super().move()} silently"
```

Interview Takeaway: Use inheritance for *IS-A* relationships where the child truly specializes the parent. Prefer **composition** when you're tempted by code reuse alone.

Polymorphism **Definition**: One interface, many forms. The same method call behaves differently depending on the actual object type at runtime.

Two flavors:

- **Compile-time (overloading)**: same method name, different signatures
- **Runtime (overriding)**: subclass replaces parent method

```
vehicles: list[Vehicle] = [Car(), Truck(), ElectricCar()]
for v in vehicles:
    print(v.move())   # each calls its own move() — runtime polymorphism
```

Interview Takeaway: Polymorphism eliminates type-checking `if isinstance(x, Car)` chains. Code against the interface; let the runtime dispatch.

SOLID Principles

Letter	Principle	One-Line Meaning	Smell It Prevents
S	Single Responsibility	A class has ONE reason to change	God class, spaghetti
O	Open/Closed	Open for extension, closed for modification	Feature additions break existing code
L	Liskov Substitution	Subtypes must be substitutable for their base type	Broken inheritance hierarchies
I	Interface Segregation	Clients shouldn't depend on methods they don't use	Fat interfaces force stub implementations

Letter	Principle	One-Line Meaning	Smell It Prevents
D	Dependency Inversion	Depend on abstractions, not concretions	Hard coupling to third-party libraries

S — Single Responsibility Principle (SRP)

CODE

```
# VIOLATION: UserManager does auth, email, AND DB — three reasons to change
class UserManager:
    def login(self, username, password): ...
    def send_welcome_email(self, user): ...
    def save_to_database(self, user): ...

# FIX: split into focused classes
class AuthService:
    def login(self, username, password): ...

class EmailService:
    def send_welcome_email(self, user): ...

class UserRepository:
    def save(self, user): ...
```

O — Open/Closed Principle (OCP)

CODE

```
# VIOLATION: adding a new shape requires editing calculate_area()
def calculate_area(shape):
    if shape.type == "circle":
        return 3.14 * shape.radius ** 2
    elif shape.type == "rectangle":
        return shape.w * shape.h
    # ... must edit this every time

# FIX: each shape knows its own area
class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...

class Circle(Shape):
    def area(self): return 3.14 * self.radius ** 2

class Rectangle(Shape):
    def area(self): return self.w * self.h
# Adding Triangle → new class only, zero edits to existing code
```

L — Liskov Substitution Principle (LSP)

CODE

```
# VIOLATION: Square overrides setter in a way that breaks Rectangle's contract
class Rectangle:
    def set_width(self, w): self._w = w
    def set_height(self, h): self._h = h
    def area(self): return self._w * self._h

class Square(Rectangle):              # Square IS-NOT a Rectangle in behavior
    def set_width(self, w):
        self._w = self._h = w         # breaks caller expectation
```

```
# FIX: don't force Square into Rectangle hierarchy;
# use a common Shape abstraction without set_width/set_height
```

LSP rule: If you find yourself throwing `NotImplementedError` or doing nothing in an overridden method, it's an LSP violation.

I — Interface Segregation Principle (ISP)

```
# VIOLATION: Robot forced to implement eat()
class Worker(ABC):
    @abstractmethod
    def work(self): ...
    @abstractmethod
    def eat(self): ...

class Robot(Worker):
    def work(self): ...
    def eat(self): raise NotImplementedError  # ISP violation

# FIX: segregate interfaces
class Workable(ABC):
    @abstractmethod
    def work(self): ...

class Eatable(ABC):
    @abstractmethod
    def eat(self): ...

class Human(Workable, Eatable):
    def work(self): ...
    def eat(self): ...

class Robot(Workable):
    def work(self): ...  # no eat() burden
```

D — Dependency Inversion Principle (DIP)

```
# VIOLATION: high-level OrderService directly depends on MySQLDatabase
class OrderService:
    def __init__(self):
        self.db = MySQLDatabase()     # concrete dependency

# FIX: depend on the abstraction
class Database(ABC):
    @abstractmethod
    def save(self, order): ...

class OrderService:
    def __init__(self, db: Database):  # injected abstraction
        self.db = db

# Caller injects MySQLDatabase() or MockDatabase() — same OrderService
```

Composition vs. Inheritance

	Inheritance	Composition
Relationship	IS-A	HAS-A
Coupling	Tight (child knows parent internals)	Loose (owns a reference)
Reuse mechanism	Inherit behavior	Delegate to component
Runtime flexibility	Fixed at compile time	Can swap component at runtime
Deep hierarchies	Brittle (”fragile base class” problem)	Flat, easy to follow

Rule: *Favor composition over inheritance* (GoF, Item 18 in Effective Java).

CODE

```
# Inheritance-heavy — brittle
class Logger(FileHandler):          # tied to file handling forever
    ...

# Composition — flexible
class Logger:
    def __init__(self, handler: LogHandler):  # swap: FileHandler, ConsoleHandler, DBHandler
        self._handler = handler

    def log(self, msg: str):
        self._handler.emit(msg)
```

When inheritance IS right: genuine IS-A + you want to expose the full parent API + hierarchy is shallow ($\leq$ 2 levels).

Coupling and Cohesion

Coupling = degree to which one module depends on another.

- **Loose coupling** = good. Changes in one class don't ripple everywhere.
- **Tight coupling** = bad. Changing `MySQLDatabase` breaks `OrderService`.

Cohesion = degree to which elements inside a module belong together.

- **High cohesion** = good. `AuthService` only does authentication.
- **Low cohesion** = bad. `UtilsManager` does logging, emailing, and caching.

Golden Rule: **High cohesion + Loose coupling = maintainable software.**

DRY / KISS / YAGNI

Principle	Expansion	Meaning	Anti-pattern it prevents
DRY	Don't Repeat Yourself	Every piece of knowledge has a single representation	Copy-paste bugs, inconsistent updates
KISS	Keep It Simple, Stupid	Prefer simple solutions over clever ones	Over-engineered abstractions
YAGNI	You Ain't Gonna Need It	Don't build features until they're needed	Speculative generality, unused code

1.7 Design Patterns

Design patterns are **named solutions to recurring design problems**. They are not libraries — they're vocabulary and blueprints.

Source: Gang of Four (GoF) — *Design Patterns: Elements of Reusable Object-Oriented Software* (1994).

Pattern Summary Table

Pattern	Category	Intent	Real-World Use
Singleton	Creational	One instance, global access	Config, Logger, Connection Pool
Factory Method	Creational	Subclass decides which object to create	Document editors, UI widgets
Abstract Factory	Creational	Family of related objects without specifying classes	Cross-platform UI toolkits
Builder	Creational	Construct complex objects step by step	SQL query builders, HTTP requests
Prototype	Creational	Clone existing objects	Undo/redo, game object spawning
Adapter	Structural	Wrap incompatible interface	Third-party library integration
Decorator	Structural	Add behavior dynamically without subclassing	Java I/O streams, middleware
Facade	Structural	Simplified interface to a subsystem	SDK wrappers, home theater
Proxy	Structural	Control access to another object	Lazy loading, caching, auth
Composite	Structural	Treat individual/groups uniformly	File systems, UI component trees
Strategy	Behavioral	Define interchangeable algorithms	Sorting, pricing, compression
Observer	Behavioral	One-to-many event notification	Event listeners, MVC, pub/sub
Command	Behavioral	Encapsulate a request as an object	Undo/redo, queued jobs, macros
State	Behavioral	Object changes behavior when state changes	Order lifecycle, traffic light
Template Method	Behavioral	Define skeleton; subclasses fill in steps	Game loops, data export pipelines
Iterator	Behavioral	Sequential access without exposing internals	Collection traversal

Creational Patterns

Singleton **Intent**: Ensure only one instance of a class exists and provide a global access point.

When to use: Logger, configuration, thread pool, connection pool — resources that must be shared and have exactly one owner.

CODE

```
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

# Thread-safe version (Python)
import threading

class ThreadSafeSingleton:
    _instance = None
    _lock = threading.Lock()

    @classmethod
    def get_instance(cls):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:       # double-checked locking
                    cls._instance = cls()
        return cls._instance
```

Real example: `java.lang.Runtime.getRuntime()`, database connection pool.

Pitfall: Hard to unit test (global state). In interviews, mention using dependency injection instead of Singleton for testability.

Factory Method **Intent**: Define an interface for creating an object, but let subclasses decide which class to instantiate.

When to use: When the exact type to create isn't known until subclass decision, or you want to decouple creation from usage.

CODE

```
class Notification(ABC):
    @abstractmethod
    def send(self, message: str): ...

class EmailNotification(Notification):
    def send(self, message): print(f"Email: {message}")

class SMSNotification(Notification):
    def send(self, message): print(f"SMS: {message}")

class NotificationFactory:
    @staticmethod
    def create(type_: str) -> Notification:
        if type_ == "email":
            return EmailNotification()
        elif type_ == "sms":
            return SMSNotification()
        raise ValueError(f"Unknown type: {type_}")

# Usage
notif = NotificationFactory.create("email")
notif.send("Welcome!")
```

Real example: `Calendar.getInstance()` in Java, Django ORM `objects.create()`.

Abstract Factory **Intent**: Create families of related objects without specifying their concrete classes.

When to use: When your system must be independent of how its products are created, and you want to enforce that products from the same family are used together.

CODE

```
# Abstract products
class Button(ABC):
    @abstractmethod
    def render(self): ...

class Checkbox(ABC):
    @abstractmethod
    def render(self): ...

# Concrete Windows family
class WindowsButton(Button):
    def render(self): print("Windows Button")

class WindowsCheckbox(Checkbox):
    def render(self): print("Windows Checkbox")

# Abstract factory
class GUIFactory(ABC):
    @abstractmethod
    def create_button(self) -> Button: ...
    @abstractmethod
```

CODE

```
    def create_checkbox(self) -> Checkbox: ...

class WindowsFactory(GUIFactory):
    def create_button(self): return WindowsButton()
    def create_checkbox(self): return WindowsCheckbox()

# Client
def render_ui(factory: GUIFactory):
    factory.create_button().render()
    factory.create_checkbox().render()
```

vs Factory Method: Factory Method creates ONE product; Abstract Factory creates a FAMILY of related products.

Builder **Intent**: Construct a complex object step by step, separating construction from representation.

When to use: When an object has many optional parameters, or its construction requires multiple steps.

CODE

```
class Pizza:
    def __init__(self):
        self.size = None
        self.crust = None
        self.toppings = []

class PizzaBuilder:
    def __init__(self):
        self._pizza = Pizza()

    def size(self, size: str):
        self._pizza.size = size
        return self           # fluent API

    def crust(self, crust: str):
        self._pizza.crust = crust
        return self

    def topping(self, t: str):
        self._pizza.toppings.append(t)
        return self

    def build(self) -> Pizza:
        return self._pizza

# Usage
pizza = (PizzaBuilder()
         .size("large")
         .crust("thin")
         .topping("mushrooms")
         .topping("olives")
         .build())
```

Real example: Java `StringBuilder`, `HttpRequest.Builder`, Lombok `@Builder`, SQL query builders.

Prototype **Intent**: Create new objects by copying (cloning) an existing object.

When to use: When object creation is expensive (DB call, heavy computation) and you want to clone a cached version.

CODE

```
import copy

class GameCharacter:
    def __init__(self, name, health, weapons):
        self.name = name
        self.health = health
        self.weapons = weapons[:]  # shallow copy of list

    def clone(self):
        return copy.deepcopy(self)

# Usage
template = GameCharacter("Warrior", 100, ["sword", "shield"])
clone1 = template.clone()
clone1.name = "Warrior-2"
```

Real example: Undo/redo in editors, spawning enemy waves in games, `Object.clone()` in Java.

Structural Patterns

Adapter **Intent**: Convert the interface of a class into another interface that clients expect. Makes incompatible interfaces work together.

When to use: Integrating third-party libraries, legacy code, or two systems with different interfaces.

CODE

```
# Target interface our system expects
class JSONLogger(ABC):
    @abstractmethod
    def log_json(self, data: dict): ...

# Third-party logger with incompatible interface
class LegacyLogger:
    def write_log(self, message: str):
        print(f"[LEGACY] {message}")

# Adapter bridges the gap
class LegacyLoggerAdapter(JSONLogger):
    def __init__(self, legacy: LegacyLogger):
        self._legacy = legacy

    def log_json(self, data: dict):
        message = str(data)
        self._legacy.write_log(message)   # translates call

# Client uses JSONLogger interface; doesn't know about legacy
adapter = LegacyLoggerAdapter(LegacyLogger())
adapter.log_json({"event": "login", "user": "alice"})
```

Real example: Java `InputStreamReader` (adapts `InputStream` to `Reader`), power plug adapters.

Decorator **Intent**: Attach additional responsibilities to an object dynamically, as an alternative to subclassing.

When to use: Adding features (logging, caching, compression, authentication) without modifying the original class.

CODE

```
class Coffee(ABC):
    @abstractmethod
    def cost(self) -> float: ...
```

CODE

```
    @abstractmethod
    def description(self) -> str: ...

class SimpleCoffee(Coffee):
    def cost(self): return 1.0
    def description(self): return "Coffee"

class MilkDecorator(Coffee):
    def __init__(self, coffee: Coffee):
        self._coffee = coffee

    def cost(self): return self._coffee.cost() + 0.25
    def description(self): return self._coffee.description() + ", Milk"

class SugarDecorator(Coffee):
    def __init__(self, coffee: Coffee):
        self._coffee = coffee

    def cost(self): return self._coffee.cost() + 0.10
    def description(self): return self._coffee.description() + ", Sugar"

# Compose dynamically
coffee = SugarDecorator(MilkDecorator(SimpleCoffee()))
print(coffee.cost())          # 1.35
print(coffee.description())   # Coffee, Milk, Sugar
```

Real example: Java I/O (`BufferedInputStream(new FileInputStream(...))`), Python `@functools.lru_cache`, HTTP middleware chains.

vs Inheritance: Decorator wraps at runtime; inheritance is fixed at compile time. Decorators compose; subclasses explode combinatorially.

Facade **Intent**: Provide a simplified interface to a complex subsystem.

When to use: When a subsystem has many moving parts and clients don't need all that complexity.

CODE

```
class DVDPlayer:
    def on(self): print("DVD on")
    def play(self, movie): print(f"Playing {movie}")

class Amplifier:
    def on(self): print("Amp on")
    def set_volume(self, v): print(f"Volume: {v}")

class Projector:
    def on(self): print("Projector on")
    def wide_screen(self): print("Wide screen mode")

# Facade hides the complexity
class HomeTheaterFacade:
    def __init__(self, dvd, amp, projector):
        self._dvd, self._amp, self._proj = dvd, amp, projector

    def watch_movie(self, movie: str):
        self._amp.on(); self._amp.set_volume(5)
        self._proj.on(); self._proj.wide_screen()
        self._dvd.on(); self._dvd.play(movie)

# Client calls ONE method
theater = HomeTheaterFacade(DVDPlayer(), Amplifier(), Projector())
theater.watch_movie("Inception")
```

Real example: `slf4j` logging facade, AWS SDK clients, REST API that wraps microservices.

Proxy **Intent**: Provide a surrogate that controls access to another object.

Flavors:

- **Virtual Proxy**: lazy initialization (create expensive object on demand)
- **Protection Proxy**: access control
- **Remote Proxy**: local handle for remote object (RPC)
- **Caching Proxy**: cache results

CODE

```
class Image(ABC):
    @abstractmethod
    def display(self): ...

class RealImage(Image):
    def __init__(self, filename: str):
        self.filename = filename
        self._load()              # expensive

    def _load(self):
        print(f"Loading {self.filename} from disk...")

    def display(self):
        print(f"Displaying {self.filename}")

class ImageProxy(Image):          # virtual proxy — defers loading
    def __init__(self, filename: str):
        self.filename = filename
        self._real = None

    def display(self):
        if self._real is None:
            self._real = RealImage(self.filename)   # load on first use
        self._real.display()
```

Real example: Spring AOP proxies, Hibernate lazy loading, `java.lang.reflect.Proxy`.

Composite **Intent**: Compose objects into tree structures to represent part-whole hierarchies. Treat individual objects and compositions uniformly.

When to use: File systems (file/folder), UI component trees, org charts.

CODE

```
class FileSystemComponent(ABC):
    @abstractmethod
    def show(self, indent=0): ...

class File(FileSystemComponent):
    def __init__(self, name: str):
        self.name = name

    def show(self, indent=0):
        print(" " * indent + self.name)

class Directory(FileSystemComponent):
    def __init__(self, name: str):
        self.name = name
        self._children: list[FileSystemComponent] = []

    def add(self, c: FileSystemComponent):
```

CODE

```
        self._children.append(c)

    def show(self, indent=0):
        print(" " * indent + f"[{self.name}]")
        for child in self._children:
            child.show(indent + 2)

# Build tree
root = Directory("root")
src = Directory("src")
src.add(File("main.py")); src.add(File("utils.py"))
root.add(src); root.add(File("README.md"))
root.show()
```

Behavioral Patterns

Strategy **Intent**: Define a family of algorithms, encapsulate each, and make them interchangeable.

When to use: Multiple ways to do the same task that should be swappable at runtime (sort orders, payment methods, compression algorithms).

CODE

```
class SortStrategy(ABC):
    @abstractmethod
    def sort(self, data: list) -> list: ...

class BubbleSort(SortStrategy):
    def sort(self, data): return sorted(data)   # simplified

class QuickSort(SortStrategy):
    def sort(self, data): return sorted(data, key=lambda x: x)

class Sorter:
    def __init__(self, strategy: SortStrategy):
        self._strategy = strategy

    def set_strategy(self, strategy: SortStrategy):
        self._strategy = strategy

    def sort(self, data: list) -> list:
        return self._strategy.sort(data)

# Usage
sorter = Sorter(BubbleSort())
result = sorter.sort([3, 1, 2])
sorter.set_strategy(QuickSort())  # swap at runtime
```

vs if-else: Strategy eliminates `if type == "bubble" ... elif type == "quick"`. Adding a new sort = new class, zero edits.

Real example: `java.util.Comparator`, payment processors, compression codecs.

Observer **Intent**: Define a one-to-many dependency so that when one object changes state, all dependents are notified automatically.

When to use: Event systems, MVC (model notifies views), pub/sub, real-time dashboards.

CODE

```
class Observer(ABC):
    @abstractmethod
    def update(self, event: str, data): ...
```

CODE

```python
class Subject:
    def __init__(self):
        self._observers: list[Observer] = []

    def subscribe(self, o: Observer):
        self._observers.append(o)

    def unsubscribe(self, o: Observer):
        self._observers.remove(o)

    def notify(self, event: str, data):
        for o in self._observers:
            o.update(event, data)

class StockMarket(Subject):
    def __init__(self):
        super().__init__()
        self._price = 0

    def set_price(self, price: float):
        self._price = price
        self.notify("price_change", price)

class StockAlert(Observer):
    def update(self, event, data):
        print(f"Alert! Stock price: {data}")

class StockDisplay(Observer):
    def update(self, event, data):
        print(f"Display updated: {data}")

market = StockMarket()
market.subscribe(StockAlert())
market.subscribe(StockDisplay())
market.set_price(150.0)   # both get notified
```

Real example: Java `EventListener`, React state management, Django signals, `Observable` in RxJava.

Command **Intent**: Encapsulate a request as an object, allowing parameterization, queuing, logging, and undo/redo.

When to use: Undo/redo, transactional operations, job queues, macro recording.

CODE

```python
class Command(ABC):
    @abstractmethod
    def execute(self): ...
    @abstractmethod
    def undo(self): ...

class TextEditor:
    def __init__(self): self.text = ""
    def insert(self, s: str): self.text += s
    def delete(self, n: int): self.text = self.text[:-n]

class InsertCommand(Command):
    def __init__(self, editor: TextEditor, text: str):
        self._editor, self._text = editor, text

    def execute(self): self._editor.insert(self._text)
    def undo(self): self._editor.delete(len(self._text))
```

CODE

```
class CommandHistory:
    def __init__(self): self._history = []

    def execute(self, cmd: Command):
        cmd.execute()
        self._history.append(cmd)

    def undo(self):
        if self._history:
            self._history.pop().undo()
```

Real example: Git commits (undo = revert), database transactions, UI action history.

State **Intent**: Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.

When to use: Order lifecycle (PENDING → PAID → SHIPPED → DELIVERED), traffic lights, vending machine.

CODE

```
class OrderState(ABC):
    @abstractmethod
    def pay(self, order): ...
    @abstractmethod
    def ship(self, order): ...

class PendingState(OrderState):
    def pay(self, order):
        print("Payment processed")
        order.state = PaidState()
    def ship(self, order):
        print("Cannot ship — not paid")

class PaidState(OrderState):
    def pay(self, order):
        print("Already paid")
    def ship(self, order):
        print("Shipping order")
        order.state = ShippedState()

class ShippedState(OrderState):
    def pay(self, order): print("Already paid")
    def ship(self, order): print("Already shipped")

class Order:
    def __init__(self): self.state = PendingState()
    def pay(self): self.state.pay(self)
    def ship(self): self.state.ship(self)

order = Order()
order.pay()    # "Payment processed"
order.ship()   # "Shipping order"
order.ship()   # "Already shipped"
```

vs if-else: State eliminates massive `if self.status == "pending": ... elif self.status == "paid"` chains. Each state is its own class.

Template Method **Intent**: Define the skeleton of an algorithm in a base class; defer specific steps to subclasses.

When to use: Algorithms with a fixed structure but variable steps (data export, game loops, test frameworks).

CODE

```
class DataExporter(ABC):
    # Template method — defines the algorithm skeleton
    def export(self, filename: str):
        data = self.fetch_data()         # step 1 — fixed
        formatted = self.format(data)    # step 2 — varies
        self.save(formatted, filename)   # step 3 — varies
        self.notify()                    # step 4 — fixed

    def fetch_data(self):
        return {"rows": [1, 2, 3]}      # common implementation

    @abstractmethod
    def format(self, data) -> str: ...

    @abstractmethod
    def save(self, content: str, filename: str): ...

    def notify(self):
        print("Export complete!")

class CSVExporter(DataExporter):
    def format(self, data): return ",".join(str(r) for r in data["rows"])
    def save(self, content, filename): open(filename, "w").write(content)

class JSONExporter(DataExporter):
    import json
    def format(self, data): return str(data)
    def save(self, content, filename): open(filename, "w").write(content)
```

vs Strategy: Template Method uses inheritance to vary steps; Strategy uses composition to swap the whole algorithm.

Iterator **Intent**: Provide a sequential access mechanism to elements of a collection without exposing the underlying structure.

CODE

```
class BookCollection:
    def __init__(self): self._books = []
    def add(self, book: str): self._books.append(book)
    def __iter__(self): return iter(self._books)   # Python built-in

library = BookCollection()
library.add("Clean Code"); library.add("Design Patterns")
for book in library:   # uses Iterator
    print(book)
```

Real example: Python `iter()`/`__iter__`, Java `Iterator<T>`, database cursors.

1.8 Architecture / Diagrams

UML Class Diagram Notation Cheat-Sheet

```
Relationship       Notation (ASCII)       Meaning
──────────────────────────────────────────────────────────────
Association        A ──────────> B         A uses/knows B
Aggregation        A ◇──────────> B        A has B (B exists independently)
Composition        A ◆──────────> B        A owns B (B dies with A)
Inheritance        A ──────────▷ B         A is-a B (extends/implements)
Dependency         A - - - - - > B         A uses B temporarily (method arg)
Realization        A - - - - - ▷ B         A implements interface B
```

UML Relationship Table

Relationship	Symbol	Meaning	Example
Association	—>	General link between classes	Student uses Library
Aggregation	◇—>	Weak whole-part (part survives)	Team has Players (players exist without team)
Composition	◆—>	Strong whole-part (part dies with whole)	House has Rooms (room has no meaning without house)
Inheritance	—▷	IS-A, extends/implements	Car extends Vehicle
Dependency	- ->	Uses temporarily (parameter/local var)	Controller depends on Service
Realization	- -▷	Implements interface	MySQLDB realizes Database

Parking Lot ASCII Class Diagram

```
+------------------+        +-------------------+
|   ParkingLot     |◆──────>|      Floor        |
+------------------+  1..*  +-------------------+
| -id: int         |        | -floorNo: int     |
| -floors: List    |        | -spots: List      |
+------------------+        +-------------------+
| +getAvailSpot()  |                 |◆ 1..*
| +assignSpot()    |                 v
+------------------+        +-------------------+
                            |   ParkingSpot     |
         +------------------+-------------------+
         |                  | -spotId: int      |
+--------+-------+          | -type: SpotType   |
|    Vehicle     |          | -isOccupied: bool |
+----------------+          +-------------------+
| -licensePlate  |          | +occupy()         |
| -vehicleType   |          | +vacate()         |
+----------------+          +-------------------+
       △
       |
   ┌───┴────┐
   |        |
+----+  +-------+
|Car |  | Truck |
+----+  +-------+

+-------------------+     +-------------------+
|    Ticket         |     |    Payment        |
+-------------------+     +-------------------+
| -ticketId: str    |     | -amount: float    |
| -entryTime: dt    |     | -paymentType      |
| -vehicle: Vehicle |     +-------------------+
| -spot: ParkingSpot|     | +process()        |
+-------------------+     +-------------------+
| +calculateFee()   |
+-------------------+
```

Multiplicity Notation

```
1       exactly one
0..1    zero or one (optional)
*       zero or more
1..*    one or more
2..5    specific range
```

1.9 Real-World Examples

Design a Parking Lot

Requirements (clarified): Multi-floor, multiple spot types (compact/large/handicapped), entry/exit gates, ticket system, payment.

Key entities: `ParkingLot`, `Floor`, `ParkingSpot` (abstract) → `CompactSpot`, `LargeSpot`, `HandicappedSpot`, `Vehicle` (abstract) → `Car`, `Truck`, `Motorcycle`, `Ticket`, `Payment`, `Gate` (Entry/Exit), `PricingStrategy`

Patterns applied:

- **Singleton**: `ParkingLot` — one instance managing the whole lot
- **Factory**: Create appropriate `ParkingSpot` type
- **Strategy**: `PricingStrategy` — hourly, flat-rate, weekend-rate
- **Observer**: Notify display boards when spot count changes

CODE

```python
from enum import Enum
from datetime import datetime

class SpotType(Enum):
    COMPACT = "compact"
    LARGE = "large"
    HANDICAPPED = "handicapped"

class ParkingSpot:
    def __init__(self, spot_id: str, spot_type: SpotType):
        self.spot_id = spot_id
        self.spot_type = spot_type
        self._is_occupied = False
        self._vehicle = None

    def is_available(self) -> bool:
        return not self._is_occupied

    def occupy(self, vehicle):
        self._is_occupied = True
        self._vehicle = vehicle

    def vacate(self):
        self._is_occupied = False
        self._vehicle = None

class Ticket:
    def __init__(self, ticket_id: str, vehicle, spot: ParkingSpot):
        self.ticket_id = ticket_id
        self.vehicle = vehicle
        self.spot = spot
        self.entry_time = datetime.now()
        self.exit_time = None

class PricingStrategy(ABC):
    @abstractmethod
    def calculate(self, hours: float) -> float: ...
```

CODE

```python
class HourlyPricing(PricingStrategy):
    def __init__(self, rate: float): self.rate = rate
    def calculate(self, hours): return hours * self.rate

class ParkingLot:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self):
        if not hasattr(self, '_initialized'):
            self._floors = []
            self._pricing = HourlyPricing(2.0)
            self._initialized = True

    def find_available_spot(self, vehicle_type) -> ParkingSpot:
        for floor in self._floors:
            for spot in floor.spots:
                if spot.is_available():
                    return spot
        return None
```

Design an Elevator System

Key entities: `ElevatorSystem`, `Elevator`, `Floor`, `Request` (internal/external), `ElevatorController`, `Door`, `Display`

Patterns:

- **State**: Elevator states — `IDLE`, `MOVING_UP`, `MOVING_DOWN`, `DOORS_OPEN`
- **Strategy**: Scheduling algorithm (FCFS, SCAN/LOOK)
- **Observer**: Floor panels observe elevator position

Core state machine:

```
IDLE → MOVING_UP → DOORS_OPEN → IDLE
     → MOVING_DOWN → DOORS_OPEN → IDLE
```

Key design decision: Should `Elevator` know about scheduling? No — use a `Scheduler` (SRP + Strategy). `Elevator` follows orders; `Scheduler` decides which requests to fulfill in what order.

Design a Vending Machine

Key entities: `VendingMachine`, `Item`, `Slot`, `CoinSlot`, `Display`, `VendingState`

Patterns:

- **State**: `IDLE`, `HAS_MONEY`, `DISPENSING`, `OUT_OF_STOCK`
- **Singleton**: `VendingMachine`
- **Command**: Each user action (insert coin, select item, cancel) as a Command

State transitions:

```
IDLE ──(insertCoin)──> HAS_MONEY ──(selectItem)──> DISPENSING ──(dispense)──> IDLE
                    ──(cancel)──> IDLE
IDLE ──(selectItem)──> "Insert coin first"
```

Design a Library Management System

Key entities: `Library`, `Book`, `BookItem` (physical copy), `Member`, `Librarian`, `Loan`, `Reservation`, `Catalog`, `Search`

Patterns:

- **Facade**: `LibraryService` wraps catalog search, loan management, member management
- **Observer**: Notify waitlisted members when a book becomes available
- **Strategy**: Search strategy (by title, author, ISBN, category)
- **Factory**: Create different `Member` types (Student, Faculty, Guest)

1.10 Real-Life Analogies

One smart home — every principle and pattern is a switch, a socket, or a system in the same house.

Concept	Real-Life Analogy
Encapsulation	The walls hiding the wiring — you flip a switch and the light comes on; you never touch the live cables behind the plasterboard
Abstraction	The thermostat dial — you set 21°C and walk away; the furnace, gas valve, and duct fans sort themselves out behind the wall
Inheritance	The ”Villa” plan extends the base ”House” plan — it inherits every room, every circuit, every load-bearing wall, and adds a pool wing on top
Polymorphism	One ”Open” button on the smart-home app that swings the front door, rolls up the garage shutter, and tilts the skylight — each mechanism moves in its own way
SRP	The smoke detector detects smoke; the siren makes noise; the sprinkler douses flames — one job per device, swap any one without touching the others
OCP	The breaker panel — you snap a new circuit breaker in to power the home office extension without touching the existing breakers that run the kitchen
LSP	Any certified light bulb — LED, halogen, or smart bulb — fits the same socket and dims when the dimmer turns; no fixture needs rewiring for the swap
ISP	The outdoor weatherproof socket only implements the ”power” interface; it knows nothing about the thermostat's heating schedule — each device gets only the interface it actually uses
DIP	Every appliance plugs into the same standard socket — an abstraction — rather than being hard-wired to the power station; swap the supplier, not the toaster
Singleton	The main fuse box — exactly one exists in the house; every circuit routes through it and the architect makes sure nothing bypasses it
Factory	The prefab workshop off-site — you tell it ”I need a kitchen module” and it delivers a fully fitted unit; you never see the assembly jigs or which crew built it
Abstract Factory	The architect's room-kit catalogue — order the ”Scandinavian” kit and every fitting (taps, handles, light switches) arrives from the same matching family
Builder	Fitting out a bare shell room step by step — lay flooring, then paint walls, then hang curtains, then mount shelves — each step optional, sequence matters, one `handOver()` at the end
Prototype	The show-home template — instead of designing each holiday-let room from scratch, you clone the master layout and tweak only the colour scheme
Adapter	A travel plug adapter on the kitchen counter — the European two-pin appliance plugs into it, and the adapter presents the UK three-pin shape the house expects
Decorator	Outfitting a bare concrete room layer by layer — insulation, then plasterboard, then paint, then coving, then smart lighting — each layer adds behaviour without demolishing what came before
Facade	The smart-home app's ”Movie Night” scene — one tap dims every light, lowers the blinds, turns on the projector, and silences the doorbell; you never touch seven separate sub-systems
Proxy	The smart doorbell that screens visitors — it answers the door, checks the camera, and only rings you inside if it can't verify the caller itself

Concept	Real-Life Analogy
Observer	The smoke detectors — one sensor in the kitchen smells smoke and instantly notifies every alarm, the sprinkler controller, and the security panel across the house
Strategy	Swappable heating behind the same thermostat dial — gas boiler today, electric heat-pump tomorrow, solar thermal next year; the dial interface never changes
Command	A scene button you programme yourself — press it once to execute a sequence of actions (dim lights, lock doors, set alarm), press again to undo, queue multiple scenes to run at midnight
State	The thermostat's mode wheel — Home, Away, and Sleep each change what temperature the whole house targets; same thermostat, entirely different behaviour per mode
Composite	The master suite — it groups the bedroom, en-suite, and dressing room as a single bookable unit, yet each sub-room also exposes the same `inspect()` interface individually
Template Method	The architect's standard commissioning checklist — always: pressure-test plumbing, then certify electrics, then sign off ventilation; the steps are fixed but each trade fills in its own method
Iterator	The electrician's walkthrough — she moves from socket to socket around every room in a fixed sequence without needing a floor-plan in her hand; the house exposes one `nextSocket()` cursor

1.11 Memory Tricks / Mnemonics

SOLID Expanded Mnemonic

”Some Old Lions Don't Invert”

Letter	Principle	Memory Hook
S	Single Responsibility	**S**niper — one target at a time
O	Open/Closed	**O**pen door, closed vault — can enter new, can't change old
L	Liskov	**L**egacy employee — should work like original without surprises
I	Interface Segregation	**I**nterface diet — no fat interfaces
D	Dependency Inversion	**D**on't call us, we'll call you (Hollywood principle)

Design Pattern Memory Aids

Creational — ”SAFBP” (So A Factory Builds Prototypes)

- **S**ingleton, **A**bstract Factory, **F**actory Method, **B**uilder, **P**rototype

Structural — ”ABCDFP” (Always Build Clean Design For People)

- **A**dapter, **B**ridge, **C**omposite, **D**ecorator, **F**acade, **P**roxy

Behavioral — ”CCIMMOST” (Clever Coders Iterate Memorably, Most Often See Tricks)

- **C**hain of Responsibility, **C**ommand, **I**terator, **M**ediator, **M**emento, **O**bserver, **S**tate, **S**trategy, **T**emplate Method, **V**isitor

Pattern "When to Use" Quick Recall

Trigger Word	Pattern
”Only one instance”	Singleton
”Create without knowing exact type”	Factory Method
”Family of related objects”	Abstract Factory
”Step by step construction”	Builder
”Clone an expensive object”	Prototype
”Incompatible interfaces”	Adapter

Trigger Word	Pattern
”Add behavior dynamically”	Decorator
”Simplify complex subsystem”	Facade
”Control access / lazy load”	Proxy
”Tree structure, treat uniformly”	Composite
”Swappable algorithms”	Strategy
”Event notification”	Observer
”Undo/redo, queue requests”	Command
”Behavior changes with state”	State
”Fixed skeleton, variable steps”	Template Method
”Sequential collection traversal”	Iterator

1.12 Common Interview Questions

Classic LLD Prompts

Prompt	Key Patterns	Important Classes
Design a Parking Lot	Singleton, Factory, Strategy, Observer	ParkingLot, Floor, Spot, Vehicle, Ticket
Design an Elevator	State, Strategy, Observer	Elevator, Request, Scheduler, State
Design a Vending Machine	State, Singleton, Command	VendingMachine, Slot, CoinSlot
Design a Library System	Facade, Observer, Strategy	Library, Book, Member, Loan
Design a Chess Game	Factory, State, Composite	Board, Piece, Player, Move
Design a Hotel Booking	Factory, Strategy, Observer	Hotel, Room, Booking, Payment
Design an ATM	State, Command, Strategy	ATM, Card, Account, Transaction
Design a Food Ordering App	Factory, Observer, Strategy	Restaurant, Menu, Order, Delivery
Design a Movie Ticket System	Factory, Strategy, Observer	Cinema, Show, Seat, Booking
Design a Ride-sharing App	Observer, Strategy, State	Driver, Rider, Trip, Location

Approach for Each

1. **Clarify**: ”Is this for a single parking lot or a chain?” / ”How many floors, concurrent users?”
2. **Entities**: List nouns → candidate classes
3. **Relationships**: IS-A vs HAS-A for each pair
4. **Core classes**: Code 2-3 most important ones fully
5. **Patterns**: Name which you used and why
6. **Extensibility**: ”If requirement X changes, I'd...”

Follow-Up Questions to Expect

- ”How would you add multi-currency support to your vending machine?”
- ”What if two users try to book the same parking spot simultaneously?” (**thread safety**)
- ”How would you scale this to 1000 elevators?” (**HLD pivot**)
- ”What if we need to support dynamic pricing?” (**Strategy pattern**)
- ”How would you test this design?” (**testability, DI**)

1.13 Senior-Level Discussion Points

When NOT to Use a Pattern

Patterns have costs. Know when to skip them:

Pattern	Don't Use When
Singleton	You need testability (inject dependencies instead)
Abstract Factory	Only one product family exists now
Observer	Notification order matters (observers execute in undefined order)
Decorator	You need to inspect the wrapped type at runtime
Command	Simple one-shot operations with no undo requirement
State	Only 2-3 states that rarely change (if-else is clearer)

Over-Engineering Warning Signs

- **Pattern for pattern's sake**: "I used Factory here so..." (but there's only ever one concrete type)
- **Premature abstraction**: Creating interfaces for classes that will never have a second implementation
- **Deep inheritance**: > 3 levels is almost always wrong
- **Anemic Domain Model**: Classes with only getters/setters and no behavior (all logic in "Service" classes)
- **God classes**: `OrderManager` that does 20 things

Extensibility vs. Simplicity Trade-off

Open/Closed is not free. Every abstraction adds indirection. Ask:

- "Will this variation point actually be varied?"
- "Is the cost of abstraction worth the likely future change?"

> **Rule of Three**: Introduce an abstraction after the third time you need to vary it. Not before.

Thread Safety in Patterns

- **Singleton**: Double-checked locking. In Java, use `volatile`; in Python, use the GIL (but be aware of async contexts).
- **Observer**: Notification callbacks must be thread-safe; consider using a message queue.
- **Command Queue**: Use `queue.Queue` (Python) or `BlockingQueue` (Java) for thread-safe command dispatch.
- **Proxy (caching)**: Cache invalidation must be synchronized.

CODE

```python
# Thread-safe Singleton (Python)
import threading

class Config:
    _instance = None
    _lock = threading.Lock()

    @classmethod
    def get_instance(cls):
        if cls._instance is None:          # first check (no lock)
            with cls._lock:
                if cls._instance is None:  # second check (with lock)
                    cls._instance = cls()
        return cls._instance
```

Dependency Injection Container Pattern

In production code, instead of hard-coding `new MySQLDB()` everywhere, use a DI container (Spring, Guice, Python's `dependency_injector`). **In interviews**, manually inject via constructor — it signals DI awareness.

1.14 Typical Mistakes Candidates Make

Mistake	Why It's Wrong	What to Do Instead
Jump straight to code	Missed requirements surface mid-code	Always clarify, then diagram
Only one class per problem	Shows shallow thinking	Identify 5–8 entities minimum
All public fields	Breaks encapsulation	Private fields + controlled access
Inheritance for code reuse	Tight coupling, brittle	Compose if no IS-A relationship
No interfaces/abstractions	Hard-codes concrete types	Define abstractions first
Name patterns without applying them	Sounds cargo-cult	Explain WHY, show how it helps
Over-pattern small problems	Shows poor judgment	Know when simple if-else is fine
Skip extensibility discussion	Misses senior signal	Always say ”if X changes, we can...”
Anemic model (all logic in services)	Not OOP thinking	Put behavior in the right object
Forget to mention thread safety	Incomplete for concurrent systems	Always ask ”is this multi-threaded?”

1.15 How This Connects to Other Topics

LLD ↔ High-Level Design (HLD)

LLD happens **inside** the boxes of HLD. In HLD you say ”Order Service”; in LLD you design what classes are inside Order Service. In interviews, LLD is usually a stepping stone to HLD discussions.

LLD ↔ Clean Code

Clean Code principles (meaningful names, small functions, no magic numbers) are the micro-level expression of SOLID/DRY. A class with perfect SOLID compliance can still be unreadable if method names are cryptic.

LLD ↔ Concurrency

Patterns need thread-safety consideration:

- **Singleton**: Must be thread-safe (double-checked locking, `volatile`)
- **Observer**: Modifying observer list while iterating is a race condition (use `CopyOnWriteArrayList` in Java)
- **Command Queue**: Natural fit for producer-consumer with `BlockingQueue`
- **Flyweight**: Shared state must be immutable or synchronized

LLD ↔ Testing

Good LLD makes testing easy:

- **DIP** → can inject mocks instead of real dependencies
- **SRP** → each class tests independently
- **Strategy** → swap test strategies without touching production code
- **Composition** → components testable in isolation

LLD ↔ Databases / ORM

- Active Record pattern (Django models): each model knows how to save itself
- Repository pattern: separates business logic from data access (better for testing)
- Unit of Work pattern: groups DB operations into a transaction

1.16 FAANG Interview Tips

1. **Start with clarification, always.** Even 2 questions shows systematic thinking. Interviewers deduct points for assumptions that lead you down wrong paths.
2. **Narrate your design decisions.** ”I'm making `Vehicle` abstract because we'll have Cars, Trucks, Motorcycles — I want to treat them uniformly in parking logic.” Silence signals uncertainty.

3. **Draw the class diagram before coding.** Code without design = red flag. Even a rough ASCII diagram on the whiteboard earns points.
4. **Name your patterns explicitly.** ”This is a Strategy pattern because...” Naming shows pattern literacy even if your code is slightly off.
5. **Show the extensibility story.** After building your design, say: ”If we needed to add electric vehicle charging spots, I'd subclass ParkingSpot → EVSpot without touching existing code — that's OCP.” This is the #1 senior-signal moment.
6. **Don't over-engineer.** Interviewers value appropriate complexity. A 2-class problem shouldn't have 10 patterns. Say ”YAGNI — I'd add X if the requirement comes.”
7. **Be specific about trade-offs.** ”I chose Composition over Inheritance here because PricingStrategy may need to change at runtime, and subclassing would require new classes for every combination.”
8. **Handle the concurrency question.** If your Singleton or Observer is used in a multi-threaded context, proactively address thread safety. At FAANG, this is expected.
9. **Code clean, not fast.** Readable variable names, meaningful method names, proper access modifiers. Interviewers read your code; clarity > speed.
10. **Amazon-specific**: Map your design to Leadership Principles. ”Dive Deep” = knowing pattern internals. ”Insist on Highest Standards” = clean code. ”Invent and Simplify” = choosing the right pattern, not the most complex one.

1.17 Revision Cheat Sheet

10-Minute Summary

OOP: Encapsulate state → Abstract interfaces → Inherit only IS-A → Polymorphism for dispatch.

SOLID: Single responsibility → Open/closed → Liskov substitution → Interface segregation → Dependency inversion.

DRY/KISS/YAGNI: No duplication → Keep simple → Don't speculate.

Composition > Inheritance: Use HAS-A when in doubt; switch components at runtime.

High cohesion + Loose coupling = maintainable code.

Interview Steps: Clarify → Entities → Relationships → Class Diagram → Patterns → Code → Extensibility.

Key Concepts at a Glance

Topic	1-Line Summary
Encapsulation	Hide state; expose behavior
Abstraction	Code to interfaces, not implementations
Inheritance	IS-A, use sparingly, max 2-3 levels
Polymorphism	Same message, different behavior
SRP	One reason to change
OCP	Extend don't modify
LSP	Subtypes must honor base contract
ISP	Small, focused interfaces
DIP	Depend on abstractions
Singleton	One instance, controlled access
Factory	Decouple creation from usage
Builder	Step-by-step complex construction
Strategy	Swappable algorithms
Observer	Event-driven notification
Command	Encapsulate requests for undo/queue

Topic	1-Line Summary
State	Behavior changes with state
Decorator	Add behavior without subclassing
Facade	Simplify complex subsystem
Adapter	Bridge incompatible interfaces
Proxy	Control access / lazy load
Composite	Tree structures, uniform treatment

Quick Cheat-Sheet Table

SOLID	Smell Prevented	One Fix
S — Single Responsibility	God class	Extract class
O — Open/Closed	Modification to add features	Polymorphism + Strategy
L — Liskov	Broken override (NotImplementedError)	Redesign hierarchy
I — Interface Segregation	Fat interface, stub methods	Split into role interfaces
D — Dependency Inversion	`new ConcreteClass()` in business logic	Constructor injection

Pattern	I Need To...	Use When
Singleton	One instance globally	Config, Logger, Pool
Factory Method	Create without knowing type	Plugin systems
Abstract Factory	Create a family of objects	Cross-platform UI
Builder	Build complex object step-by-step	Many optional fields
Prototype	Clone expensive objects	Object pools, game spawn
Adapter	Use incompatible API	3rd party integration
Decorator	Add behavior at runtime	Middleware, I/O streams
Facade	Hide subsystem complexity	SDK wrappers
Proxy	Control/delay/cache access	Lazy load, auth, cache
Composite	Treat tree nodes uniformly	File system, UI trees
Strategy	Swap algorithms at runtime	Sorting, pricing, routing
Observer	Notify dependents on change	Events, MVC, pub/sub
Command	Queue/undo/log requests	Undo, task queues
State	Change behavior with state	Order, elevator, traffic
Template Method	Fix skeleton, vary steps	Export pipeline, game loop
Iterator	Traverse collection uniformly	Collections, cursors

Most Important Concepts (Rank-Ordered for FAANG)

1. **OOP Four Pillars** — Must articulate with examples in 30 seconds each
2. **SOLID** — Must know all 5, show violation and fix
3. **Strategy Pattern** — Most commonly applicable; almost every LLD uses it
4. **Observer Pattern** — Notification systems appear in nearly every design
5. **Singleton (thread-safe)** — Appears in most system designs; know the pitfalls
6. **Factory Method / Abstract Factory** — Object creation is always present
7. **Composition vs. Inheritance** — A litmus test for OOP maturity
8. **Builder** — Shows up in complex domain objects

9. **State Pattern** — Critical for lifecycle-heavy systems (orders, elevators)
10. **The LLD Interview Framework** — The 7-step process; follow it every time

Study tip: Design at least 3 systems (parking lot, elevator, library) from scratch without notes. The muscle memory of identifying entities, drawing class diagrams, and selecting patterns is what makes you fluent under interview pressure.